

to the entire house. If you're sceptical, try it yourself. Spin up a container and check the user.

```
docker run -it ubuntu bash
whoami
```

Nine times out of ten, you'll see the *root*. This means any vulnerability in that container has the potential to escalate privileges to the host level if it exploits a kernel flaw. Docker's architecture just isn't built for zero-trust environments or situations where security is non-negotiable. You can stack on AppArmor, SELinux, seccomp, and all the security frameworks you want, but these are band-aids, not solutions.

Linus's argument

Just so I am clear, Linus Torvalds can, in no way, be classified as anti-Docker. He does see potential in this technology, and he is right to do so. Docker has made life ultra simple for nimble programming and deployment of software across devices and platforms. Having said that, Linus has been pretty vocal about the fact that Docker's reliance on shared kernels isn't true virtualisation.

Linus was quick to criticise Docker's dependence on user namespaces as a supposed solution to privilege isolation. In theory, user namespaces allow containers to map root privileges to non-root privileges on the host, which sounds like a solid security measure. But in practice it's a leaky bucket. If you've got a process running as *root* inside a container, even with user namespaces mapping it to a *non-root* user on the host, a kernel exploit still has the potential to bypass this and gain root access on the host machine. We discussed this at length in the previous section.

Linus has also criticised how Docker containers share kernel modules and libraries with the host. Any vulnerability in these shared resources potentially opens up every container to exploitation. Docker tries to handle

this with layered filesystems, but that's more like a patchwork solution, something that someone as meticulous as Linus can never be fond of. This becomes even more of a concern when we start layering multiple containers on a single host, which is standard in most deployments. One crack in that foundation, and you're dealing with a full-scale breach.

I am not trying to put words in Linus Torvald's mouth, but I get the sense that his argument is more akin to the idea that Docker is pretending to be something it's not. VMs, with their hypervisors and dedicated kernels, offer the kind of security that Docker just can't replicate. And when it comes to environments where isolation is crucial, Linus would argue that Docker is simply out of its depth. He isn't entirely wrong. Docker's isolation model is weak compared to VMs, and its dependence on shared kernel resources is a ticking time bomb in high-stakes environments. But does that mean Docker's worthless? Not at all. It's a matter of understanding the risks and using Docker where it makes sense, not pretending it's a one-size-fits-all solution.

Something better than Docker

To wrap this whole thing up, I would like to quickly suggest some potential solutions. I titled this section as 'Something better than Docker' because I understand that some of these solutions might require a ground-up rebuild of Docker itself which might be as unideal as it might be impractical. So think of this section as an ideal repository for building something that has the benefits of Docker, but then also mitigates the potential risks. A perfect *containeriser* if you will (if there ever was one). If we want to build something that truly outclasses Docker, we need to address its weaknesses head-on and rethink containerisation from the ground up.

1. First off, we ditch the shared-kernel approach entirely. We

need to build a micro-hypervisor model, where each container runs its own minimal kernel. This ensures that every container is genuinely isolated, similar to a lightweight VM but without the bloat. By employing a microkernel architecture, you're essentially granting each container its own mini-OS that only loads essential components, drastically reducing the attack surface. This step eliminates the primary flaw of Docker's shared-kernel model.

2. Next, leverage hardware-assisted virtualisation like Intel VT-x or AMD-V to handle isolation efficiently. This is where we'll differentiate ourselves from Docker's reliance on namespaces. With hardware support, each container will get near-native performance while maintaining strict separation. For example, instead of binding everything to a Linux kernel, containers will interact directly with hardware-level isolation, meaning exploits won't have the chance to jump from one container to another.
3. We can't ignore orchestration. Rather than bolting on security later, build an orchestration layer that enforces strict security policies from the get-go. This orchestration tool, think Kubernetes but with security baked in, will enforce seccomp, AppArmor, and SELinux profiles automatically based on container configurations. For instance, before launching a container, the orchestration layer could analyse its dependencies and generate a security profile dynamically, ensuring that each container only has access to the resources it needs.
4. Let's go beyond the crude *root* vs *non-root* distinction Docker offers. Implement a permission system that assigns containers fine-grained capabilities, like capabilities management in modern OSes. You'll create an RBAC model that defines precisely what a container can or cannot access such as network resources, storage, specific hardware, etc. Imagine having a declarative YAML file that